Raytracing using the GPU
Advanced computer architecture

(B-KUL-JLI1QS)

May 15, 2024



Session Date:	May 17, 2024
Partners:	Emma Dejagere
	Brecht Van de Sijpe
Class:	MElsw
Instructor:	Alex Marinsek

1 Introduction

We implement a ray tracer on the GPU. We start from an implementation on the CPU and expand this further with CUDA. This report gives a brief overview of the principles used and the differences between the CPU and GPU implementations.

2 Method

Ray tracing is used to generate 3D images. In the beginning, ray tracing was used to render still CGI images, because it took a relatively long time to render the images. Lately however, hardware acceleration for real-time ray tracing has become common in new commercial graphics cards. Which allows for more applications in regards to motion pictures and video games.

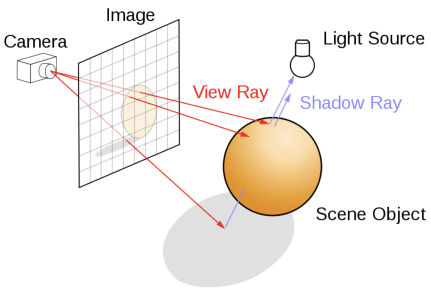


Figure 1: Basic setup ray-tracing

In general, the ray tracer uses the following workflow (1):

1 Define the objects and light sources within the scene.

2 Set up the viewing coordinate system (the synthetic camera).

3 For each pixel on the camera screen, set up a ray that starts in the camera center, and goes through the pixel.

4 Find the intersections of this ray with objects in the scene.

5 Identify the intersection point that is closest to the viewer (it is the smallest value of t for which an intersection is found). This step is however not implemented in our case.

6 Determine color and intensity of light along the ray from the intersection point. In our model this is done by calculating the cosine of the angle between the normal vector, pointing perpendicular outside on the surface on the sphere, and the ray.

As a first step, we implement a ray tracer on the CPU. For this version we inspire our design based on a tutorial by Peter Shirley (2). This tutorial gives us a basic understanding of how spheres are generated, how lighting works on ray tracing and how we could generate shadows. We build on this for our CUDA-version. Most calculation could be reused.

Initially, we used the SDL-library for rendering purposes. Because this library does not allow parallelisation and we were not rendering moving scenes, this brought unnecessary difficulties. We decided to transition into a version that exports the result to a ppm file, like we had seen in a previous lab.

To parallelize this process on the GPU, we assign a single thread to do the rendering of a single pixel in the image. The image is represented as a one dimensional array, containing three color

values (red, green, blue) for each pixel. Because of this reason, the layout of the threads in the blocks will be one dimensional too.

3 Results

Figures 2, 3 and 4 show the results when generating a single sphere in combination with different scenario's of lighting. When multiple light sources are provided, we show the location of each source with a small sphere on the location of each light source. Further on, we generate additional images, adding a bit more complexity to the scenes in figures 5 and 6. To compare performance

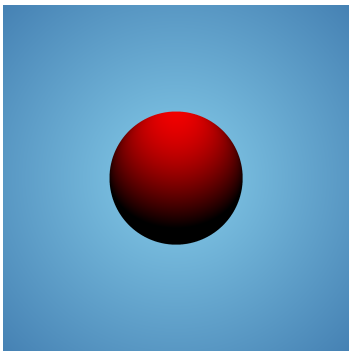


Figure 2: One sphere with lighting from the above

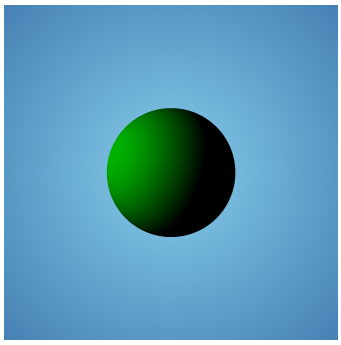


Figure 3: One sphere with lighting from the side

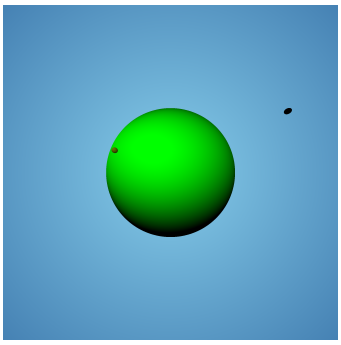


Figure 4: One sphere with multiple lighting sources

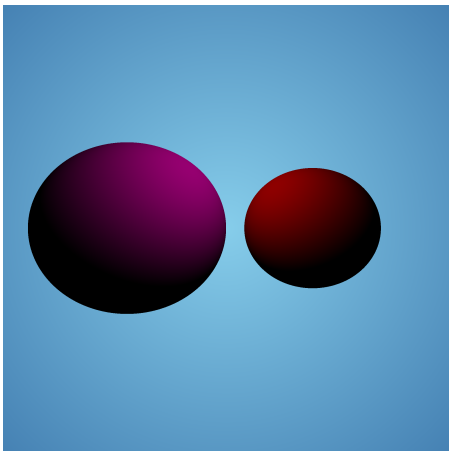


Figure 5: Two spheres, one lighting source from above

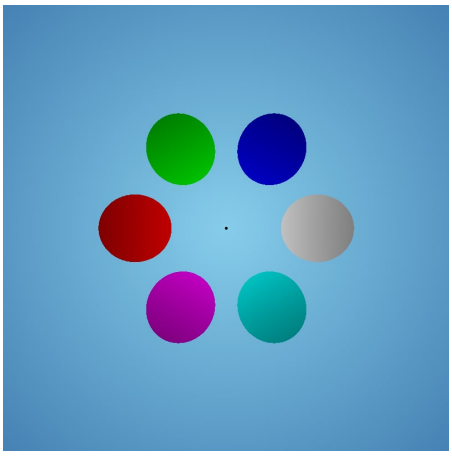


Figure 6: Six spheres, one lighting source in the center

between CPU and GPU versions, we consider a setup that has a considerable amount of spheres and lights. We increment the image size each benchmark. The algorithm has a big O complexity of $O(P * S * L)$, with P the amount of pixels, S the amount of spheres and L the amount of light sources. The results of the benchmarks can be found in table 1 and figure 7. We use a AMD Ryzen 5 4500 @3.60 GHz and NVIDIA GeForce RTX 3060 for the benchmarking.

#	Image resolution (pixels x pixels)	amount of spheres	amount of light sources	CPU time [ms]	GPU time [ms]	CPU / GPU ratio
1	200 x 200	75	66	176	0.126097	1395
2	400 x 400	75	66	708	0.317094	2232
3	800 x 800	75	66	2808	1.043433	2691
4	1600 x 1600	75	66	11386	3.773729	3017
5	3200 x 3200	75	66	45667	14.474307	3155
6	6400 x 6400	75	66	185165	56.750305	3263
7	12800 x 12800	75	66	755853	224.449768	3368

Table 1: Benchmarks comparing CPU and GPU timing

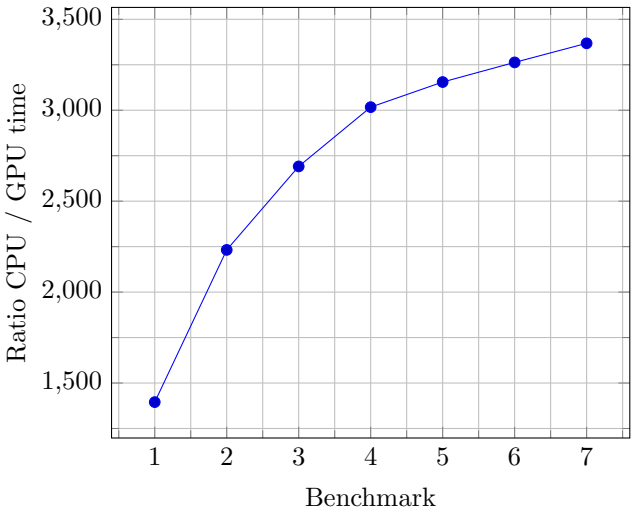


Figure 7: Ratio CPU / GPU times per benchmark

4 Conclusion

We see a big reduction in execution time when using the GPU, compared to the CPU by assigning a separate core to each pixel in the view port.

References

[1] S. Beauchemin, “Ray Tracing Principles.” [Online]. Available: {https://www.csd.uwo.ca/~sbeauche/CS3388/CS3388-Ray-Tracing-Principles.pdf}

[2] P. Shirley, “Ray Tracing in One Weekend.” [Online]. Available: {https://raytracing.github.io/books/RayTracingInOneWeekend.html.}